



---

# Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

|                    |                                          |
|--------------------|------------------------------------------|
| NIM                | 71251235                                 |
| Nama Lengkap       | NATALIA GRECIA PUTRI                     |
| Minggu ke / Materi | 07 / Percabangan dan Perulangan Kompleks |

PROGRAM STUDI INFORMATIKA  
FAKULTAS TEKNOLOGI INFORMASI  
UNIVERSITAS KRISTEN DUTA WACANA  
YOGYAKARTA  
2026

## BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

### Struktur Percabangan Kompleks

Percabangan kompleks adalah kondisi dimana sebuah program tidak hanya mengevaluasi satu kondisi, tetapi dapat memiliki banyak alternatif pilihan sekaligus. Selain itu, instruksi yang dijalankan pada setiap cabang pun dapat terdiri dari lebih dari satu perintah.

Dalam Python, percabangan kompleks umumnya dibangun menggunakan struktur if yang bersarang (nested if). Struktur ini memungkinkan program untuk menerima kondisi tambahan di dalam blok kondisi yang sudah ada sebelumnya.

#### 1. Percabangan Bertingkat (If)

Percabangan bertingkat digunakan ketika ada kondisi tambahan yang perlu dicek setelah kondisi awal terpenuhi.

Contoh:

```
1 - if kondisi1:
2 -     if kondisi2:
3 -         #Perintah jika kondisi1 dan kondisi2 terpenuhi
4 -     else:
5 -         #Perintah jika kondisi1 terpenuhi tetapi kondisi2 tidak
6 - else:
7 -     #Perintah jika kondisi1 tidak terpenuhi
```

Gambar 7.1: Percabangan bertingkat (if)

Dalam struktur ini jika kondisi pertama (kondisi1) benar, maka perintah akan lanjut mengecek kondisi kedua (kondisi2). Jika kondisi2 juga benar, maka perintah di dalamnya akan dieksekusi. Jika tidak, program menjalankan perintah dalam bagian else.

#### 2. Percabangan dengan Elif (Else If)

Percabangan dengan elif digunakan ketika terdapat banyak kondisi yang mungkin terjadi, dan hanya satu yang akan dieksekusi.

Contoh:

```
1 ▾ if kondisi1:
2     #Perintah jika kondisi 1 terpenuhi
3 elif kondisi2
4     #Perintah jika kondisi2 terpenuhi
5 ▾ elif kondisi3:
6     #Perintah jika kondisi3 terpenuhi
7 ▾ else:
8     #Perintah jika tidak ada kondisi yang terpenuhi
```

Gambar 7.2: Percabangan elif

Struktur ini berguna untuk skenario yang membutuhkan banyak kondisi tetapi hanya memilih satu yang paling sesuai.

### 3. Kombinasi Percabangan (If, Elif, Else)

Program membutuhkan kombinasi dari if, elif, dan if bertingkat agar dapat menangani kondisi yang lebih kompleks.

Contoh:

```
1 ▾ if kondisi1:
2 ▾     if kondisi2:
3         #Perintah jika kondisi1 dan kondisi2 terpenuhi
4 ▾     elif kondisi3:
5         #Perintah jika kondisi1 dan kondisi3 terpenuhi
6 ▾     else:
7         #Perintah jika hanya kondisi1 yang terpenuhi
8 ▾ else:
9 ▾     if kondisi4:
10        #Perintah jika kondisi4 terpenuhi
11 ▾     else:
12        #Perintah jika tidak ada kondisi yang terpenuhi
```

Gambar 7.3: Kombinasi Percabangan (If, Elif, Else)

Dalam struktur ini, terdapat dua bagian utama if kondisi1 dan else. Jika kondisi1 benar, maka program akan mengecek kondisi-kondisi lain di dalamnya. Jika tidak, maka program akan masuk ke else dan mengecek kondisi lain.

## Struktur Perulangan Kompleks

### Break

Perintah break digunakan untuk menghentikan proses perulangan yang sedang berlangsung sebelum mencapai batas akhirnya. Biasanya perintah ini dikombinasi dengan struktur if untuk menentukan kapan perulangan harus dihentikan.

Hal penting yang harus diperhatikan adalah posisi perintah print relatif terhadap break. Jika print diletakkan sebelum break, maka nilai terakhir (nilai yang memicu break) masih akan ditampilkan. Sebaliknya jika print diletakkan setelah break, nilai tersebut tidak akan ditampilkan.

Contoh break dalam perulangan for:

Walaupun perulangan ditetapkan hingga 1000, program berhenti di angka 5 karena break.

Contoh break dalam perulangan while:

```
1  for i in range(1000):
2      print(i)
3      if i == 10:
4          break
```

Gambar 5.: break dalam perulangan for

Output:

```
0
1
2
3
4
5
6
7
8
9
10
```

Gambar 5.: output break

Walaupun perulangan ditetapkan hingga 1000, program berhenti di angka 10 karena break.

Contoh break dalam perulangan while:

```
1  while i < 100:
2      print(i)
3      if i == 10:
4          break  #keluar dari loop jika i = 10
5      i += 1
```

Gambar 7.: break dalam while

### Continue

Perintah continue tidak menghentikan perulangan secara keseluruhan, melainkan hanya melewati satu iterasi tertentu dan langsung melanjutkan ke iterasi berikutnya. Semua perintah yang terletak setelah continue pada iterasi tersebut tidak akan dieksekusi.

Contoh continue dalam for:

```
1  for i in range(10):
2      if i == 5:
3          continue
4      print(i)
```

Gambar 7.: continue dalam for

Ketika  $i == 5$ , perintah `print(i)` akan dilewati, sehingga angka 5 tidak akan muncul.

```
0
1
2
3
4
6
7
8
9
```

Gambar 7.: output continue

### Perulangan Bertingkat

Perulangan kompleks juga dapat berarti perulangan didalam perulangan (loop).

Contohnya:

```
1  for i in range(1, 4): #outer loop
2      for j in range(1, 4): #inner loop
3          print(f"({i}, {j})", end=" ")
4      print()
```

Gambar 7.: contoh perulangan bertingkat

Dalam contoh ini:

- Outer loop(i) berjalan 3 kali.
- Inner loop(j) berjalan 3 kali untuk setiap perulangan i.

Output:

```
(1, 1) (1, 2) (1, 3)
(2, 1) (2, 2) (2, 3)
(3, 1) (3, 2) (3, 3)
```

Gambar 7.: output perulangan bertingkat

Contoh Perulangan Kompleks yang lain:

Menampilkan pola segitiga angka menggunakan perulangan bersarang:

```
1  n = 5
2  for i in range(1, n+1):
3      for j in range(1, i+1):
4          print(i, end=" ")
5      print()
```

Gambar 7.: perulangan kompleks lain

- Outer loop mengontrol jumlah baris.
- Inner loop mengontrol angka yang dicetak pada setiap baris.

Output:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Gambar 7.: output perulangan bersarang

## BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

### SOAL 1

[https://github.com/grec12/71251235\\_nataliagrecia.git](https://github.com/grec12/71251235_nataliagrecia.git)

```
1 def prim(v):
2     if v < 2:
3         return False
4     for i in range(2, int(v**0.5) + 1):
5         if v % i == 0:
6             return False
7     return True
8
9 def prima_terdekat(n):
10    if n <= 2:
11        prim(f"Tidak ada bilangan prima yang leboh kecil dari {n}")
12        return
13    for i in range(n - 1, 1, -1):
14        if prim(i):
15            return i
16
17 n = int(input("Masukkan niai v: "))
18 print(prima_terdekat(n))
```

Penjelasan Kode:

#### 1. Fungsi prim(v) – (Mengecek Bilangan Prima)

```
1 def prim(v):
2     if v < 2:
3         return False
4     for i in range(2, int(v**0.5) + 1):
5         if v % i == 0:
6             return False
7     return True
```

- Jika  $v < 2$ , maka akan langsung mengembalikan False karena bukan prima.
- Loop dari 2 sampai akar kuadrat dari  $v$  untuk mengecek apakah ada pembagi.
- Jika  $v \% i == 0$ , berarti  $v$  dapat dibagi  $i$
- Jika tidak ada pembagi yang ditemukan, maka akan mengembalikan True (bilangan prima).

#### 2. Fungsi prima\_terdekat(n) – (Mencari Prima Terkecil di bawah n)

```

9  def prima_terdekat(n):
10     if n <= 2:
11         prim(f"Tidak ada bilangan prima yang lebih kecil dari {n}")
12         return
13     for i in range (n - 1, 1, -1):
14         if prim(i):
15             return i

```

- Jika  $n \leq 2$ , maka tidak ada bilangan prima yang lebih kecil dan akan menampilkan pesan dan berhenti.
- `Range(n - 1, 1, -1)`: loop mundur dari  $n - 1$  turun ke 2.
- Setiap angka  $i$  dicek menggunakan fungsi `prim(i)`.
- Begitu ditemukan bilangan prima pertama (terbesar dibawah  $n$ ), maka akan langsung `return i`.

### 3. Program Utama – (Input Output)

```

17  n = int(input("Masukkan nilai v: "))
18  print(prima_terdekat(n))

```

- `Input()` membaca teks dari pengguna, lalu `int()` mengubahnya menjadi bilangan bulat.
- Hasil dari `prima_terdekat(n)` langsung dicetak ke layar.

Output:

```

Masukkan nilai v: 12
11
PS E:\latihan> & C:/Anaco
Masukkan nilai v: 21
19

```

## SOAL 2

```

1  import math
2  n = int(input("Masukkan nilai n: "))
3
4  for i in range(n, 0, -1):
5      faktorial = math.factorial(i)
6      print(faktorial, end=" ")
7      for j in range(i, 0, -1):
8          print(j, end=" ")
9      print()

```

Penjelasan Kode:

1. Import Model math

```
1 import math
```

Mengimpor modul math agar dapat menggunakan fungsi matematika, seperti `math.factorial()`.

2. Menerima Input dari Pengguna

```
2 n = int(input("Masukkan nilai n: "))
```

Meminta pengguna memasukkan nilai integern.

3. Perulangan for untuk Menampilkan Faktorial dari Pola Angka

```
4 for i in range(n, 0, -1):
```

Looping dari n hingga 1 dengan langkah -1 (mundur).

4. Menghitung Faktorial

```
5 faktorial = math.factorial(i)
6 print(faktorial, end=" ")
```

- `math.factorial(i)`: Menghitung faktorial dari i.
- `faktorial, end=" "`: Mencetak faktorial i dalam satu baris.

5. Perulangan Kedua untuk Pola Angka

```
7 for j in range(i, 0, -1):
8     print(j, end=" ")
```

- Looping dari i hingga 1 dengan langkah -1.
- Mencetak angka dari i ke 1 dalam satu baris.

6. Pindah ke Baris Baru

```
9 print()
```

Berfungsi untuk berpindah ke baris baru setelah mencetak satu pola angka.

Output:

```
Masukkan nilai n: 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

### SOAL 3

```
1 def cetak_deret(tinggi, lebar):
2     angka = 1
3     for i in range(tinggi):
4         baris = []
5         for j in range(lebar):
6             baris.append(str(angka))
7             angka += 1
8         print(" ".join(baris))
9
10    tinggi = int(input("Masukkan tinggi: "))
11    lebar = int(input("Masukkan lebar: "))
12
13    cetak_deret(tinggi, lebar)
```

Penjelasan code:

1. Fungsi cetak\_deret(tinggi, lebar)

```
1 def cetak_deret(tinggi, lebar):
```

Fungsi ini digunakan untuk mencetak pola angka dalam bentuk persegi panjang dengan jumlah baris tinggi dan jumlah.

2. Inisialisasi Variabel Angka

```
2     angka = 1
```

Variabel angka diinisialisasi dengan nilai 1 dan akan bertambah setiap kali angka ditambahkan ke dalam baris.

3. Perulangan untuk Mencetak Pola

```
3     for i in range(tinggi):
```

Looping i untuk mengontrol jumlah baris sesuai dengan tinggi.

4. Membuat List Baris

```
4         baris = []
```

List baris digunakan untuk menyimpan angka dalam satu baris.

5. Perulangan Kedua untuk Mengisi Baris

```
5         for j in range(lebar):
6             baris.append(str(angka))
7             angka += 1
```

- Looping j untuk mengontrol jumlah angka dalam satu baris sesuai dengan lebar.

- `baris.append(str(angka))`: Menambahkan angka ke dalam list baris.
- `angka += 1`: Menambahkan nilai angka setiap kali angka ditambahkan.

#### 6. Mencetak Baris

```
8      print(" ".join(baris))
```

Menggabungkan elemen list baris menjadi satu string dengan spasi sebagai pemisah dan mencetak.

#### 7. Meminta Input dari Pengguna

```
10     tinggi = int(input("Masukkan tinggi: "))
11     lebar = int(input("Masukkan tinggi: "))
```

Memanggil fungsi `cetak_deret` untuk mencetak nilai tinggi dan lebar.

#### 8. Memanggil Fungsi `cetak_deret(tinggi,lebar)`

```
13     cetak_deret(tinggi, lebar)
```

Memanggil fungsi `cetak_deret` untuk mencetak pola angka.

Output:

```
Masukkan tinggi: 5
Masukkan tinggi: 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```